

1st REVIEW

Student Course Enrollment and Academic Management System

V. ABHIDEV	2520030325
TEJ PRATAP SINGH	2520030476
G. SHASHANK	2520030495

Under the Guidance
of

Dr. Y Subbarayudu

Assistant Professor
Department of Computer Science and Engineering



KONERU LAKSHMAIAH EDUCATION FOUNDATION

Team Members

V. ABHIDEV

2520030325

TEJ PRATAP SINGH

2520030476

G. SHASHANK

2520030495

Abstract

The system centralizes student course enrollment and academic records. The same database supports student and administrator views.

Students select a cluster, choose compatible sections and review their timetable. Four clusters contain section groups arranged as 2, 3, 2 and 3.

The backend validates eligibility and available seats before saving enrollment. Checks include prerequisites, the 24-credit limit and timetable conflicts.

Administrators manage student courses, marks, attendance and official CGPA. Official CGPA changes require an administrative reason.

React connects to a Node.js/Express REST API backed by MySQL. REST endpoints return JSON responses for the interface.

Introduction

Course registration requires consistent information about students, sections and seats. Course sections carry schedules, instructors and capacity information.

Manual coordination makes it difficult to check prerequisites and timetable conflicts. Students need clear reasons when a registration request fails.

A centralized portal gives students and administrators access to the same records. Students view personal data while administrators manage authorized records.

Backend rules enforce registration decisions even when requests bypass the interface. The API checks each request before executing database writes.

Persistent academic records support later review and reporting. Stored results also support academic summaries and rankings.

Existing Process and Limitations

Spreadsheet-based records require manual updates across separate files. Repeated edits can leave student and course records inconsistent.

Registration checks can depend on staff reviewing each student's eligibility. A missing prerequisite or overlapping class can escape a manual check.

Separate grade and enrollment records can make academic tracking difficult. Administrators need to relate published grades to the registered courses.

A frontend-only application cannot reliably enforce seats or access permissions. API validation protects the same rules when a client sends a direct request.

Administrative changes need a traceable history for later review. Audit records help explain who changed an academic value and why.

Problem Statement

Prevent duplicate registrations and enrollment beyond section capacity. The database must preserve valid student and course relationships.

Reject courses with missing prerequisites or timetable conflicts. Students receive an explanation when the requested registration is invalid.

Keep each student's course selections within one semester cluster. A locked choice applies to every subject in the same semester.

Restrict academic changes to authorized administrators. Student requests cannot perform administrator-only operations.

Maintain consistent course, grade and attendance records in the database. Related changes must succeed together or roll back together.

Proposed Solution

Build a React portal with separate student and administrator workflows. The portal presents personal academic details and administrative summaries.

Use Express routes and services to enforce academic rules. Routes handle HTTP requests and services apply enrollment decisions.

Store related academic records in MySQL with foreign keys and constraints. The schema separates accounts, offerings, enrollments and academic results.

Use transactions and row locks for enrollment and waitlist promotion. A released seat triggers another eligibility check before promotion.

Protect requests with JWT authentication and role-based permissions. Passwords use salted hashes and protected requests carry a Bearer token.

Project Objectives

Provide a complete course enrollment workflow with persistent records. Students can select sections and inspect the saved timetable.

Enforce cluster consistency, prerequisites and the semester credit limit. Validation runs on the backend for every enrollment request.

Support timetable generation and academic progress tracking. Published results produce GPA values weighted by course credits.

Give administrators grade, attendance and official CGPA controls. The directory connects each student with their enrolled subjects.

Document REST endpoints and prepare repeatable API tests in Postman. The OpenAPI contract describes supported endpoints and request structures.

System Modules

Module	Responsibility
Authentication	Login, JWT verification and student/admin permissions
Course enrollment	Catalog, sections, prerequisites, seats and waitlists
Clusters	Four clusters, section groups and locked student selections
Academic records	Grades, credit-weighted GPA and attendance
Administration	Student directory, official CGPA and academic change history

Software Requirements

Layer or tool	Selected technology	Purpose
Frontend	React and Vite	Student and administrator pages
Backend	Node.js and Express	REST APIs and business rules
Database	MySQL with mysql2	Relational storage and transactions
API testing	Postman	Prepared requests and response checks
Development	VS Code and GitHub	Editing, execution and version control
Database inspection	MySQL Workbench	SQL queries and stored-record inspection

Conclusion and Future Scope

The project provides a consistent backend for enrollment and academic administration. The student and admin workflows share persistent relational records.

Server validation and transactions protect student registration records. Rejected requests leave no partial enrollment in the database.

Cluster rules and role permissions organize student and administrator workflows. Official CGPA updates include a reason and academic change history.

Future work includes institutional login integration and operational backups. Backups should include tested restoration before wider deployment.

Deployment automation and workload testing can support a later production release. Future testing should cover Windows, MySQL and full browser workflows.

Expected Outcomes and Evaluation Evidence

Expected outcome	Evidence for evaluation
Validated enrollment	Successful registration and a rejected incompatible request
Consistent cluster choice	My Cluster selection and a cross-cluster rejection
Academic administration	Student directory, enrolled subjects and official CGPA update
Persistent records	MySQL queries show the records used by the interface
Traceable operations	Enrollment audit and academic change history
Reproducible project	GitHub source, README, schema and setup instructions

Project Documentation and Demonstration Plan

Deliverable	Content and use
Review presentations	Problem definition, implementation and final project outcomes
README.md	Configuration, startup, credentials guidance and demo sequence
server/db/schema.sql	Relational schema, constraints, views and stored database logic
server/db/seed.js	Sample accounts, course offerings and academic records
Postman collection	Prepared requests for login, catalog, clusters and protected APIs
Live demonstration	Student action, administrator action and matching SQL record

Repository: github.com/Tejpratap-069/dbse-dbd-projects Folder: Project

THANK YOU

Questions and Live Demonstration